Karnataka Residential Educational Institutions Society

ICT Class 9 - Summative Assessment 1

Summative Assessment (SA-1) - Answer Key

Class: Class 9

Assessment Type: Summative Assessment (SA-1)

Total Marks: 40

Duration: 2 hours

Topics Covered: Python Fundamentals, Data Structures, SQL, Web Technologies

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Part A: MCQ (10 x 1 = 10 marks)

1. **Answer:** a) `[1, 3, 5, 2, 4]`
   - **Explanation:** `x[::2]` gives elements at indices 0,2,4: `[1,3,5]`. `x[1::2]` gives indices 1,3: `[2,4]`. Concatenation gives `[1,3,5,2,4]`.
2. **Answer:** a) `dict = {"a": 1, "b": 2}`
   - **Explanation:** Valid dictionary creation uses curly braces with key-value pairs. Option B creates a list of tuples. Option C uses a list as key (invalid). Option D is invalid syntax.
3. **Answer:** b) `"hon"`
   - **Explanation:** `s[-3:]` takes the last 3 characters from "Python": "hon".
4. **Answer:** c) `DROP TABLE`
   - **Explanation:** `DROP TABLE` removes the table structure and all data permanently. `DELETE` removes rows, `TRUNCATE` removes all rows but keeps structure.
5. **Answer:** b) `<ul>`
   - **Explanation:** `<ul>` defines an unordered (bulleted) list. `<ol>` is for ordered lists, `<li>` defines list items.
6. **Answer:** b) O(log n)
   - **Explanation:** Binary search repeatedly halves the search space, giving O(log n) time complexity on a sorted list.
7. **Answer:** b) `font-family`
   - **Explanation:** `font-family` CSS property specifies the font for an element. E.g., `font-family: Arial, sans-serif;`.
8. **Answer:** b) `[1, 2, 3]`
   - **Explanation:** `d.values()` returns the dictionary values. `list()` converts them to a list: `[1, 2, 3]`.
9. **Answer:** b) `document.getElementById()`
   - **Explanation:** `document.getElementById()` selects a single element by its unique ID attribute.
10. **Answer:** b) Counts all rows including NULL values
    - **Explanation:** `COUNT(*)` counts every row in the table, including those with NULL values.

Part B: Short Answer (8 x 2 = 16 marks)

1. **Write a function `merge_dicts(d1, d2)` that merges dictionaries, adding values for common keys.**
   - **Answer:**

```python
def merge_dicts(d1, d2):
    result = d1.copy()
    for key, value in d2.items():
```

```
            result[key] = result.get(key, 0) + value
        return result

print(merge_dicts({"a": 1, "b": 2}, {"b": 3, "c": 4}))
# Output: {'a': 1, 'b': 5, 'c': 4}
```

- **Marking:** 1 mark for function logic, 1 mark for correct output with test case.

2. **Write the output of the diagonal extraction code.**
   - **Answer:** Output: `[1, 5, 9]`
   - **Explanation:** `lst[i][i]` extracts diagonal elements: `lst[0][0]=1`, `lst[1][1]=5`, `lst[2][2]=9`.
   - **Marking:** 1 mark for correct output, 1 mark for explanation of diagonal index logic.
3. **Write SQL queries for `Employees(emp_id, name, department, salary, joining_date)`.**
   - **Answer:**

```
-- Employees who joined in the last 2 years
SELECT * FROM Employees WHERE joining_date >= DATE_SUB(CURDATE(), INTERVAL 2 YEAR);

-- Second highest salary
SELECT MAX(salary) FROM Employees WHERE salary < (SELECT MAX(salary) FROM
Employees);

-- Departments where average salary > 50000
SELECT department, AVG(salary) AS avg_salary
FROM Employees GROUP BY department HAVING AVG(salary) > 50000;
```

   - **Marking:** 1 mark per query (3 queries = partial, 1.5 marks each rounded to 2).
4. **Differentiate between DELETE, TRUNCATE, and DROP.**
   - **Answer:**
     - `DELETE`: DML command, removes specific rows with WHERE clause, can be rolled back.
     - `TRUNCATE`: DDL command, removes all rows, faster than DELETE, cannot be rolled back in most DBs.
     - `DROP`: DDL command, removes the entire table structure and data permanently.
   - **Marking:** 1 mark for any two correct distinctions, 1 mark for examples.
5. **Explain CSS Flexbox and create a three-column layout.**
   - **Answer:**

```
.container {
    display: flex;
}
.side {
    flex: 1;
}
.center {
    flex: 2;
}
```

   - **Explanation:** Flexbox is a one-dimensional layout model. `flex: 1` gives equal width to side columns, `flex: 2` gives the center column double width.
   - **Marking:** 1 mark for explaining Flexbox, 1 mark for correct CSS code.
6. **What is responsive web design? Explain three techniques.**
   - **Answer:** Responsive web design ensures web pages render well on all devices. Three techniques:

1. Media queries: Apply different CSS based on screen size.
2. Flexible grids: Use relative units (%, vw) instead of fixed pixels.
3. Flexible images: Use `max-width: 100%` to prevent overflow.

- **Marking:** 1 mark for definition, 1 mark for any three techniques with brief explanation.

7. **Write a Python program to implement a stack using a list.**
   - **Answer:**

```python
stack = []

def push(item):
    stack.append(item)

def pop():
    if not is_empty():
        return stack.pop()
    return "Stack is empty"

def peek():
    if not is_empty():
        return stack[-1]
    return "Stack is empty"

def is_empty():
    return len(stack) == 0

def display():
    print("Stack:", stack)

push(10)
push(20)
push(30)
display()       # [10, 20, 30]
print(peek())   # 30
print(pop())    # 30
display()       # [10, 20]
```

   - **Marking:** 1 mark for stack operations, 1 mark for proper implementation with test.

8. **Write a SQL query to find the nth highest salary (n=5).**
   - **Answer:**

```sql
SELECT DISTINCT salary
FROM Employees
ORDER BY salary DESC
LIMIT 1 OFFSET 4;
```

   - Alternative using subqueries:

```sql
SELECT MAX(salary) FROM Employees
WHERE salary NOT IN (
    SELECT DISTINCT salary FROM Employees
    ORDER BY salary DESC LIMIT 4
);
```

   - **Marking:** 1 mark for LIMIT/OFFSET approach, 1 mark for subquery approach or explanation.

Part C: Long Answer (4 x 3 = 12 marks)

1. **Write a Library Management System program.**
   - **Answer:** Key components:
     - Dictionary to store books: `{"isbn": {"title": ..., "author": ..., "copies": ..., "issued": []}}`
     - Functions: `add_book()`, `issue_book()`, `return_book()`, `search_book()`, `calculate_fine()`
     - Error handling for invalid ISBN, unavailable books, overdue returns
     - Fine calculation: Rs 5 per day overdue
   - **Marking:** 1 mark for data structure design, 1 mark for core functions, 1 mark for error handling and fine calculation.

2. **Write SQL queries for the E-Commerce Database.**
   - **Answer:**

```sql
-- Top 5 customers by spending
SELECT c.name, SUM(o.total_amount) AS total_spent
FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id
GROUP BY c.cust_id, c.name ORDER BY total_spent DESC LIMIT 5;

-- Products never ordered
SELECT p.name FROM Products p
WHERE p.prod_id NOT IN (SELECT DISTINCT prod_id FROM Order_Items);

-- Monthly revenue
SELECT MONTH(order_date) AS month, SUM(total_amount) AS revenue
FROM Orders WHERE YEAR(order_date) = YEAR(CURDATE())
GROUP BY MONTH(order_date) ORDER BY month;

-- Most popular category
SELECT p.category, SUM(oi.quantity) AS total_qty
FROM Order_Items oi JOIN Products p ON oi.prod_id = p.prod_id
GROUP BY p.category ORDER BY total_qty DESC LIMIT 1;

-- Customers with more than 3 different products
SELECT c.name, COUNT(DISTINCT oi.prod_id) AS products_ordered
FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id
JOIN Order_Items oi ON o.order_id = oi.order_id
GROUP BY c.cust_id, c.name HAVING COUNT(DISTINCT oi.prod_id) > 3;
```

   - **Marking:** 1 mark for any three correct queries, 2 marks for all five correct.

3. **Create a Student Report Card Generator web page.**
   - **Answer:** Key components:
     - HTML form with all required fields using CSS Grid layout
     - JavaScript validation (marks 0-100)
     - Grade calculation (A:90+, B:75-89, C:60-74, D:45-59, F:below 45)
     - Styled report card display below form
     - Print button using `window.print()`
   - **Marking:** 1 mark for HTML form and CSS Grid, 1 mark for JavaScript logic, 1 mark for report card display and print.

4. **Write a Contact Book program using file handling.**
   - **Answer:** Key components:
     - Dictionary of lists for contacts storage
     - File operations: `json.dump()` / `json.load()` for persistence

- Operations: add, search (partial match), update, delete, display
- Search by name, phone, or email
- Sort alphabetically or by date
- CSV export using `csv` module

- **Marking:** 1 mark for data structure and file handling, 1 mark for CRUD operations, 1 mark for search, sort, and export features.

Part D: Application Based (2 x 1 = 2 marks)

1. **Student Attendance Tracking System.**
   - **Answer:**
     - Python: Dictionary with date keys, list of student names for present/absent.
     - SQL: Tables `Students(roll, name)`, `Attendance(roll, date, status)` with foreign key.
     - Web Interface: HTML table for daily view, JavaScript to filter by date/student, CSS for styling.
   - **Marking:** 1 mark for Python and SQL components, 1 mark for web interface components.
2. **Online Bookstore Inventory System.**
   - **Answer:**
     - SQL: `Books(isbn, title, author, price, stock)` with proper constraints.
     - Python: Read CSV with `csv.reader()`, validate data, insert into database.
     - Web Form: HTML form to add books, JavaScript validation, POST to server.
     - Common Query: `SELECT * FROM Books WHERE stock < 5` for low stock alert.
   - **Marking:** 1 mark for SQL schema and common query, 1 mark for Python and web components.